

Spark Broadcast Variables - What are they and how do I use them

Written by Bill Chambers on 10/09/2015.

Table of Contents

- 1. What are Broadcast Variables?
- 2. Simple Use Case
- 3. Using Broadcast Variables
- 4. More References:

Tweet (<https://twitter.com/share>)  

1 What are Broadcast Variables?

Broadcast variables are pretty simple in concept. They're variables that we want to share throughout our cluster. However there are a couple of caveats that are important to understand. Broadcast variables have to be able to fit in memory on one machine. That means that they definitely should NOT be anything super large, like a large table or massive vector. Secondly, broadcast variables are immutable, meaning that they cannot be changed later on. This may seem inconvenient but it truly suits their use case. If you need something that can change, I'd certainly point you to accumulators which will be covered in another post.

So now we know that broadcast variables are:

- Immutable
- Distributed to the cluster
- Fit in memory

Within those constraints, what are broadcast variables used for?

2 Simple Use Case

As referenced in the Apache Spark documentation, (<http://spark.apache.org/docs/latest/tuning.html#broadcasting-large-variables>) broadcast variables are a great case for "static look up tables". So what does that even mean? It means small tables that might have some metadata about one of your

tables. For example, Imagine that I've got an app like Foursquare but only works in San Francisco. I want to include the names of the neighborhoods of the checkins however I obviously don't want to store all those strings for every single checkin for reason that should be relatively obvious.

The checkin table might look like this:

UserIdNeighborhood

234 1

567 2

234 3

532 2

Then my neighborhoods table would look like:

NeighborhoodIdName

1 Mission

2 SOMA

3 Sunset

4 Haight Ashbury

Now the thing is, the checkin table is going to be **huge**. They're transactional events and with our bajillions of users all checking in - we're going to get a lot of them. So performing a standard join is going to take forever because of our little friend, the shuffle (<http://sparktutorials.net/the-simplest-explanation-of-and-approaches-to-optimizing-spark-shuffles>). Since our neighborhood table is going to be really quite small, the smarter thing to do is to ship around that small table to each node in the cluster (next to the large amount of data that will be expensive to move). Once we get it there we should perform lookups against it to join them together. To optimize this we can use a broadcast variable!

3 Using Broadcast Variables

One of the really cool thing about broadcast variables is that in Spark, they're handled by a torrent-like protocol. What happens is that the nodes in the cluster all try to distribute the variable as quickly and efficiently as possible by downloading what they can and uploading what they can. This makes them much faster than one node having to try and do everything and push the data to all nodes.

Now I'm sure you're sick of all the writing so let's get to some code! From the shell we'll run?

```
val hoods = Seq((1, "Mission"), (2, "SOMA"), (3, "Sunset"), (4, "Haight Ashbury"))
val checkins = Seq((234, 1), (567, 2), (234, 3), (532, 2), (234, 4))
val hoodsRdd = sc.parallelize(hoods)
val checkRdd = sc.parallelize(checkins)
```

```
hoods = ((1, "Mission"), (2, "SOMA"), (3, "Sunset"), (4, "Haight Ashbury"))
checkins = ((234, 1), (567, 2), (234, 3), (532, 2), (234, 4))
hoodsRdd = sc.parallelize(hoods)
checkRdd = sc.parallelize(checkins)
```

Now that we've set those up, we need to broadcast the first table.

```
val broadcastedHoods = sc.broadcast(hoodsRdd.collectAsMap())
```

```
broadcastedHoods = sc.broadcast(hoodsRdd.collectAsMap())
```

Now that that's out there across our cluster, let's go ahead and join the two!

```
val checkinsWithHoods = checkRdd.mapPartitions({row =>
  row.map(x => (x._1, x._2, broadcastedHoods.value.getOrElse(x._2, -1)))
}, preservesPartitioning = true)
```

```
rowFunc = lambda x: (x[0], x[1], broadcastedHoods.value.get(x[1], -1))
def mapFunc(partition):
  for row in partition:
    yield rowFunc(row)

checkinsWithHoods = checkRdd.mapPartitions(mapFunc,
preservesPartitioning=True)
```

Now let's make sure that we're getting what we expect!

```
checkinsWithHoods.take(5)
// res3: Array[(Int, Int, Any)] =
// Array((234,1,Mission), (567,2,SOMA), (234,3,Sunset), (532,2,SOMA), (234,4,Haight Ashbury))
```

```
checkinsWithHoods.take(5)
# [(234, 1, 'Mission'), (567, 2, 'SOMA'), (234, 3, 'Sunset'), (532, 2, 'SOMA'), (234, 4, 'Haight Ashbury')]
```

You may have noticed that whole "preserve partitioning argument and that's to prevent the shuffle of data! And with that we've concluded the simple and effective ways that broadcast variables can simplify what you're doing in Spark! Please feel free to comment below with any questions!

4 More References:

Josh Rosen of Databricks has a great Stackoverflow post on this subject! (<http://stackoverflow.com/questions/17621596/spark-whats-the-best-strategy-for-joining-a-2-tuple-key-rdd-with-single-key-rd>)

Questions or comments?

comments powered by Disqus (<http://disqus.com>)

Follow Us For Free Spark Tutorials



Via RSS (</feed.xml>)



Via Feedly

(<http://cloud.feedly.com/#subscription%2Ffeed%2Fhttp%3A%2F%2Fwww.sparktutorials.net%2Ffeed.xml>)

Or Via Email:

Email Address *

First Name

Subscribe

*no spam.
unsubscribe any time.*

Apache®, Apache Spark, Spark® (<http://spark.apache.org/>), and the spark logo are either registered trademarks or trademarks of the Apache Software Foundation (<http://www.apache.org/>) in the United States and/or other countries. The Apache Software Foundation has no affiliation with and does not endorse or review the materials provided at this website.